Актуальные зарплаты в IT
110k 100k 150k 120k 180k

88.16
Рейтинг
Surf
Создаём веб- и мобильные приложения
Подписаться

Surf_Studio 21 окт 2022 в 11:20

Адаптируем Android-приложение для незрячих людей. Часть 2: Jetpack Compose

8 мин 2.8K
Блог компании Surf, Программирование*, Разработка мобильных приложений*, Разработка под Android*, Accessibility*

Accessibility, или доступность, — важная штука в разработке программного обеспечения, особенно под мобильные платформы. В августе 2022 года я написал статью про Accessibility в Android. Тогда я рассказал про имплементацию Accessibility в системе View, но не сказал ни слова про поддержку модного и молодёжного Jetpack Compose.

А поговорить есть о чём: сделать графический интерфейс приложения на Compose доступным для людей с ограниченными возможностями стало проще, чем на View-интерфейсе.

> Рекомендую сначала ознакомиться с первой частью. Вы узнаете про Accessibility в Android, службу специальных возможностей TalkBack и познакомитесь с хорошими практиками адаптации приложения для пользователей с ограниченными возможностями. Без понимания этих моментов будет сложнее понять, что происходит в этой статье.

Чек-лист доступности приложения

Для начала давайте вспомним, что нужно, чтобы приложение стало доступным для людей с ограниченными возможностями.

- Приложение просканировано с помощью Accessibility Scanner и найдены проблемные места.
- Шрифты — только в sp-юнитах.
- Сложные компоненты с текстом имеют нефиксированный размер: пользователь может увеличить размер шрифта по системе.
- Заголовки помечены с помощью `accessibilityHeading = true`, чтобы пользователь мог перемещаться по ним через TalkBack.
- Соблюдены цветовые контрасты: нет мешанины цветов, контент не сливается с фоном.
- Не используется КАПС: на некоторых девайсах TalkBack может зачитать такой текст как аббревиатуру. Это некорректно.
- Минимальный размер кликабельных элементов — 48x48dp.
- `ContentDescription` указан там, где нужно. Если нужно, чтобы TalkBack игнорировал элемент, задан параметр `importantForAccessibility = no`.
- Использован Live Regions, если необходимо зачитывать динамически обновляемую на UI информацию.
- Проверен порядок навигации между компонентами.
- Проверено, насколько удобно пользоваться сложными кастомными компонентами.

Семантическое дерево

Графический интерфейс на Jetpack Compose рисуется с помощью @Composable-функций. Их совокупность называется **деревом композиции**: грубо говоря, это интерфейс, который мы видим на экране смартфона.

Чтобы понимать, как работает Accessibility в Compose, следует разобраться с термином **семантическое дерево**. Оно описывает графический интерфейс альтернативным способом: содержит информацию о *семантике*, то есть *что это за компонент* и *что с ним можно сделать*. Для Accessibility Services и Testing Framework это более понятно.

> Accessibility Services — утилиты, которые упрощают работу со смартфоном для людей с ограниченными возможностями. К ним относятся TalkBack, Switch Access, Voice Access и так далее. Важно не путать их с AccessibilityService — одноимённым сервисом в Android.

В документации есть хорошая схема семантического дерева.

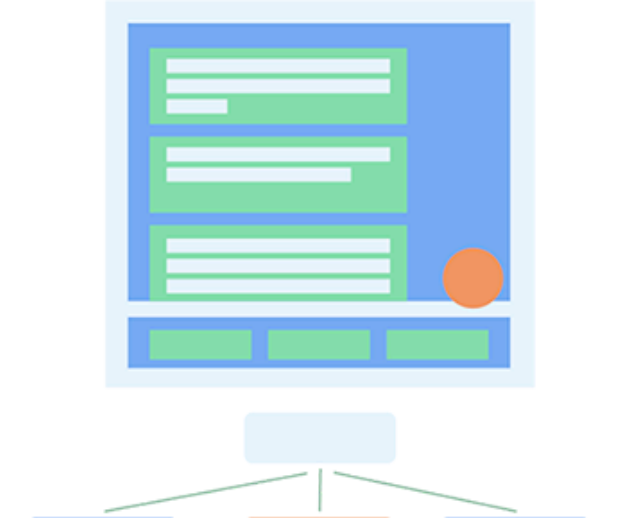


Схема семантического дерева

Сверху — видимый UI: обычный список элементов с Floating Action Button и Navigation Bar. Снизу — семантическое дерево, где каждый лепесток графа — это компонент и его семантика. TalkBack будет проходить по всем нодам дерева и зачитывать семантическую информацию.

Как сделать UI на Compose доступным

Кликабельные элементы

Пользователю с ограниченными возможностями **нужно** знать, можно сделать что-то с элементом или нет. Чтобы компонент в Compose стал кликабельным, не нужно никаких сложных конструкций: достаточно установить модификатор `Modifier.clickable` и передать туда лямбду. Это будет выглядеть примерно так.

```
Box(
    modifier = Modifier.clickable(onClickLabel = "Do something") {
        // Do something
    }
) {
   Text(text = "OK")
}
```

Если вдруг доступа к модификатору `clickable` нет, можно назначить слушателя на событие нажатия через `Modifier.semantics`.

```
Box(
    modifier = Modifier.semantics {
        onClick(label = "Do something") {
            // Do something
            true
        }
    }
) {
    Text(text = "OK")
}
```

Модификатор semantics

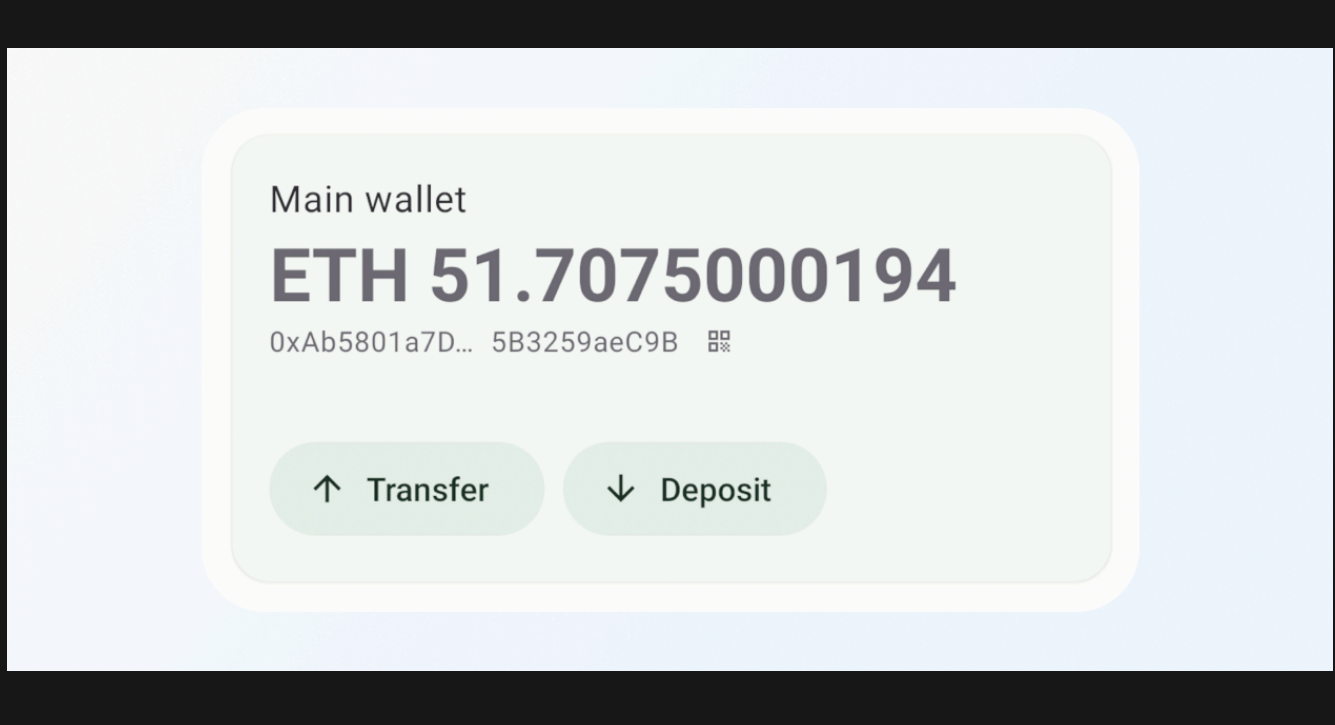
В `Modifier.semantics` обычно указывают семантическую информацию о компоненте: что за компонент, что с ним можно сделать и какое у него состояние. `Modifier.clearAndSetSemantics` создан, чтобы очистить семантику у дочерних Composable-компонентов и перегрузить её.

В обеих функциях передаётся объект `SemanticsPropertyReceiver`, который мы можем редактировать, но **не можем считывать оттуда данные**.

onClickLabel

TalkBack при наведении на кнопку зачитает «*Button, Double tap to **activate***». Если нужно дать больше семантики компоненту, можно заменить слово «*activate*» на любое другое.

В качестве примера возьмём карточку с данными о криптокошельке. Его мы разбирали в прошлой статье. При наведении на карточку слово «activate» не даст никакого контекста. Если заменить формулировку на более осмысленную, получится достаточно user-friendly: «*Card, ETH wallet, Double tap to **view wallet details***».

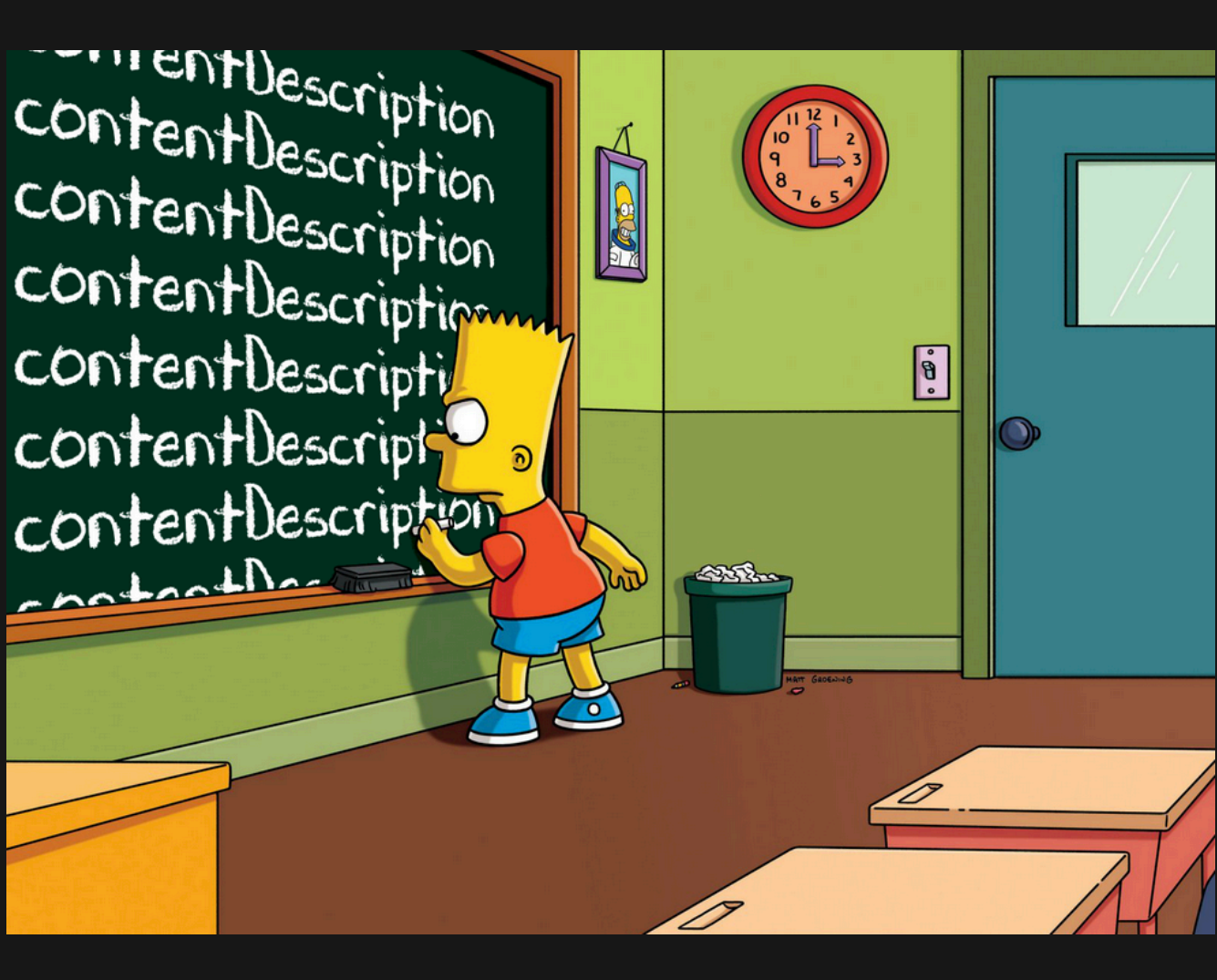


Main wallet
ETH 51.7075000194
0xAb5801a7D… 5B3259aeC9B

Информация о компоненте

У View в Android есть три основных семантических параметра:

- **contentDescription** — описание контента, находящегося в компоненте.
- **stateDescription** — описание состояния элемента: *enabled/disabled*, *checked/unchecked* и так далее.
- **roleDescription** — роль компонента на UI.



Android развивается, способы построения UI тоже, а `contentDescription` остается: его указывают для изображений или сложных компонентов.

У функции Image параметр `contentDescription` является обязательным. Можно положить туда `null`, и это будет аналогом `android:importantForAccessibility="no"`.

```
Image(
    imageVector = ImageVector.vectorResource(R.drawable.ic_image),
    contentDescription = stringResource(R.string.image_content_description)
)
```

Если нужно задать описание не изображению, используем всё тот же `Modifier.semantics` и обновляем данные в поле `SemanticsProperties.contentDescription`:

```
val desc = stringResource(R.string.component_content_description)
Box(
        modifier = Modifier.semantics {
            contentDescription = desc
  }
) {
    // Some content
}
```

Через semantics точно так же указывается и описание состояния вместе с ролью:

```
val isEnabled = true
val enabledDesc = stringResource(R.string.component_state_enabled)
val disabledDesc = stringResource(R.string.component_state_disabled)
Box(
    modifier = Modifier.semantics {
        stateDescription = if (isEnabled) {
            enabledDesc
        } else {
            disabledDesc
        }
        role = Role.Button
    }
) {
    // Some content
}
```

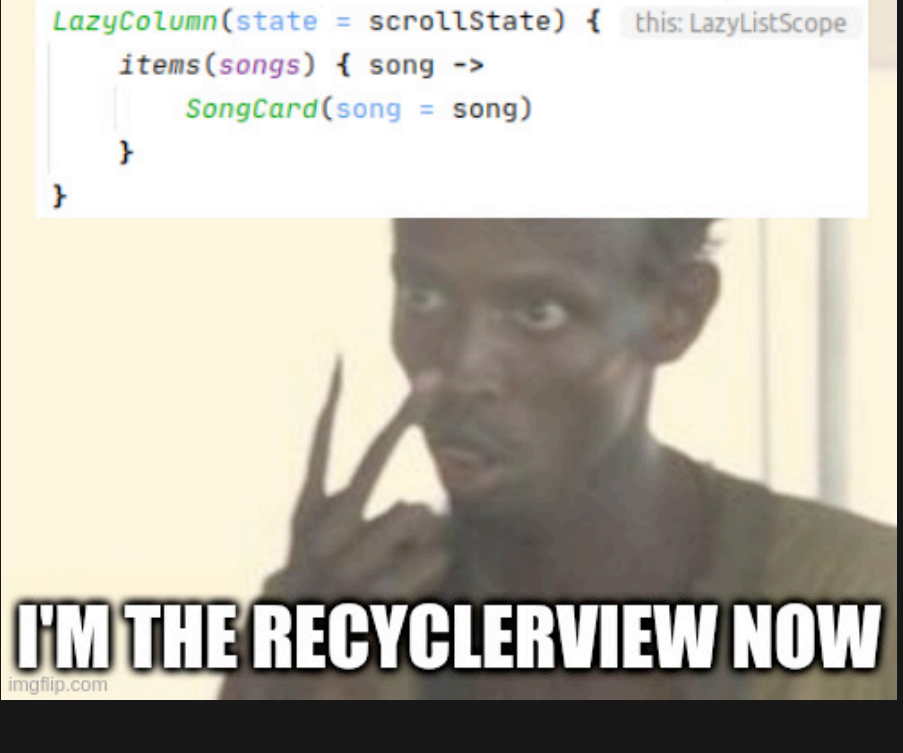
Заголовки

Заголовки нужны для того, чтобы у пользователя была возможность удобной навигации по экрану с большим количеством информации. TalkBack имеет несколько режимов навигации по компонентам, но мы затронем только навигацию по заголовкам. Чтобы сделать компонент заголовком, нужно назначить ему через модификатор `Modifier.semantics` семантику заголовка:

```
Text(
    modifier = Modifier.semantics {
        heading()
    },
    text = "Heading 1"
)
```

Списки

Поддержка Accessibility в системе View хороша тем, что для списка на `RecyclerView` не нужно задавать никакую семантику: всё работает из коробки. Наводимся на элемент списка, и TalkBack зачитывает, что это за элемент списка, какой он по счёту и какого размера список.



```
val scrollState = rememberLazyListState()
LazyColumn(state = scrollState) {  this: LazyListScope
    items(songs) { song ->
        SongCard(song = song)
    }
}
```

В Compose такой функциональности из коробки нет: придётся делать самим. Для этого есть два семантических свойства: `collectionInfo` и `collectionItemInfo`. Свойство `collectionInfo` содержит информацию о размере списка или таблицы: количество строк и столбцов. Для обычного списка количество столбцов по умолчанию равно 1. Установить `collectionInfo` нужно в модификатор semantics компонента списка.

```
Column(
    modifier = Modifier.semantics {
        collectionInfo = CollectionInfo(versions.size, 1)
    }
) {
    . . .
}
```

Свойство `collectionItemInfo` указывается на **каждый** элемент списка. Параметры `rowIndex` и `columnIndex` отвечают за позицию элемента в списке. Параметры `rowSpan` и `columnSpan` больше используются в таблицах и отвечают за количество элементов, охваченных ячейкой таблицы. Для простого списка эти параметры использовать не будем: установим значение 1.

```
versions.forEachIndexed { index, version ->
    Text(
modifier = Modifier
             .semantics {
                collectionItemInfo = CollectionItemInfo(
                    owIndex = index,
                    rowSpan = 1,
                    columnIndex = 1,
                    columnSpan = 1
                )
            }
            .fillMaxWidth()
                    .padding(16.dp),
        text = version
    )
}
```

С версии Compose 1.2.0 и выше это работает и для имплементаций `LazyList`.

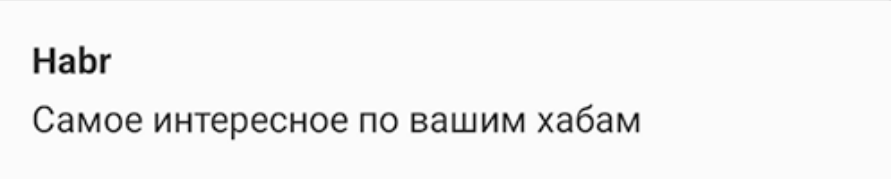
```
LazyColumn(
        modifier = Modifier.semantics {
            collectionInfo = CollectionInfo(versions.size, 1)
  }
) {
    itemsIndexed(
        items = versions,
        key = { _, version -> version.hashCode() }
    ) { index: Int, version: String ->
        Text(
            modifier = Modifier
                .semantics {
                    collectionItemInfo = CollectionItemInfo(
                            rowIndex = index,
                            rowSpan = 1,
                            columnIndex = 0,
                            columnSpan = 1
                    )
                }
                .fillMaxWidth()
                .padding(16.dp),
            text = version
        )
    }
}
```

Кастомные действия

Кастомные действия нужны для сложновыполнимых действий на UI: например, свайпов. Если у пользователя включен TalkBack, свайпнуть можно, но это достаточно неудобно. Кроме того, TalkBack бывает включён наряду вместе со Switch Access: там свайп вообще невозможен, поскольку пользователь не прикасается к экрану.

Давайте для примера разберём, как реализовать функциональность swipe-to-dismiss для людей с ограниченными возможностями. Сверстаем обычный элемент списка из приложения почтового клиента.

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
) {
    Text(
        modifier = Modifier.padding(bottom = 4.dp),
        text = "Habr",
        fontWeight = FontWeight.Bold
    )
    Text(text = "Самое интересное по вашим хабам")
}
```

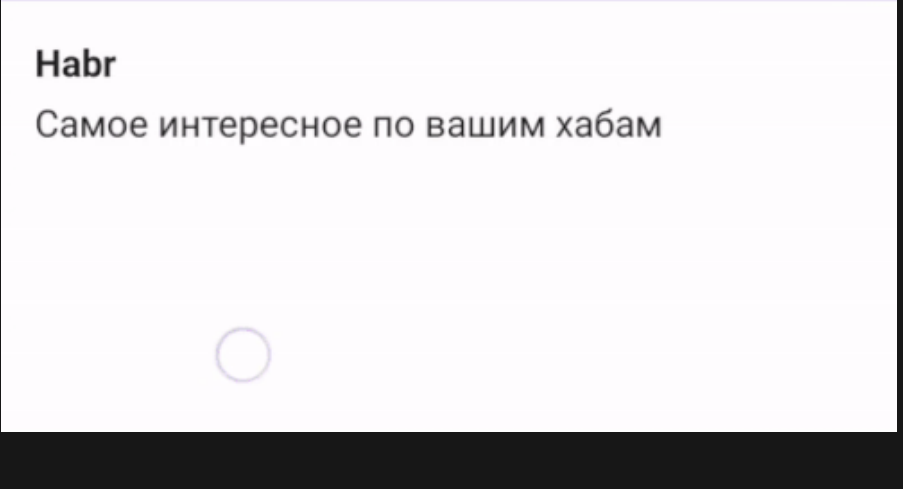


Habr
Самое интересное по вашим хабам

Простой элемент списка с заголовком и описанием

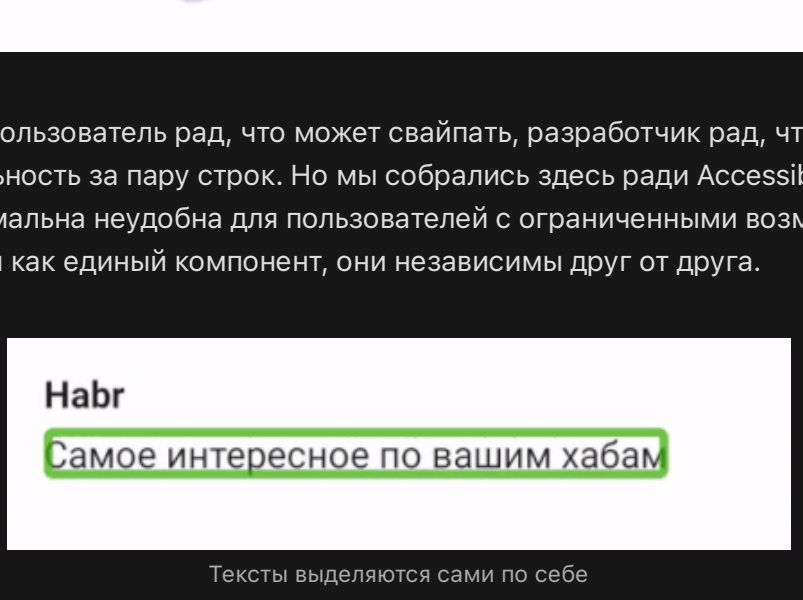
Поскольку UI верстаем через Compose, с реализацией swipe-to-dismiss не надо сильно париться: будем использовать стандартную из библиотеки. Оборачиваем вью в функцию `SwipeToDismiss`: теперь можем свайпать компонент ?

```
val state = rememberDismissState()
SwipeToDismiss(
        state = state,
        background = {}
) {
        // Здесь — Column с контентом
}
```



Habr
Самое интересное по вашим хабам

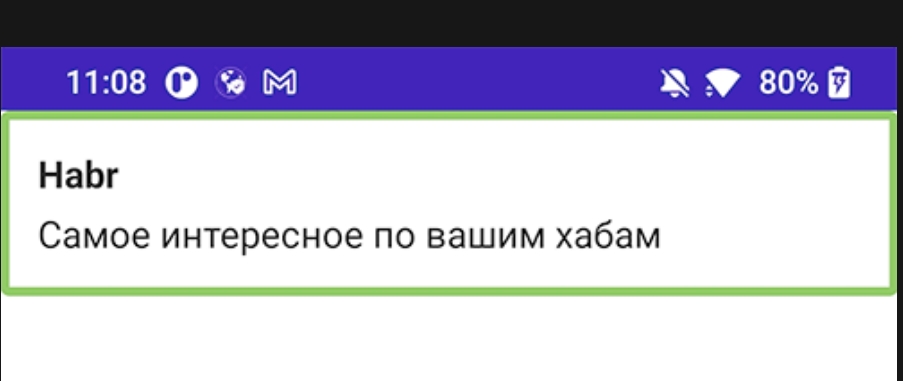
И всё бы хорошо: пользователь рад, что может свайпать, разработчик рад, что может сделать такую функциональность за пару строк. Но мы собрались здесь ради Accessibility, а эта реализация максимально неудобна для пользователей с ограниченными возможностями: тексты не читаются как единый компонент, они независимы друг от друга.



Habr
Самое интересное по вашим хабам

Тексты выделяются сами по себе

Эту проблему можно решить, установив значение `true` в модификаторе `semantics`, аргумент `mergeDescendants`. Это означает, что мы замёржили все дочерние элементы этого компонента и их невозможно выделить через TalkBack. Теперь выделяться будет весь элемент списка. То же самое можно было сделать на View, пометив все дочерние View параметром `importantForAccessibility = no`.



Habr
Самое интересное по вашим хабам

Единый компонент: TalkBack зачитывает всю информацию из него

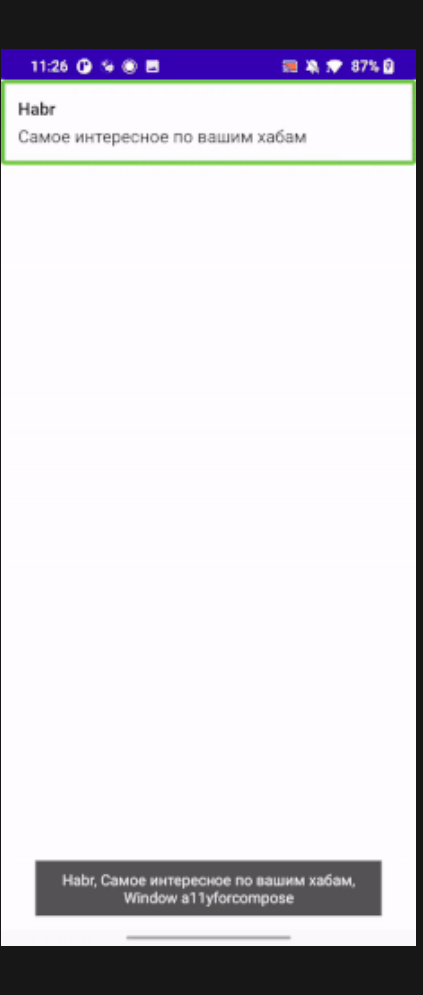
Теперь кастомизируем swipe-to-dismiss для людей с ограниченными возможностями, чтобы можно было открывать меню с дополнительными действиями и выбирать нужный элемент в списке через TalkBack.

Проинициализируем `CoroutineScope` и достанем из ресурсов локализованную строку. Напоминаю, что **все строковые ресурсы из приложения**, используемые для Accessibility, **обязаны быть локализованными**.

```
val scope = rememberCoroutineScope()
val deleteString = stringResource(R.string.delete)
SwipeToDismiss(
    . . .
)
```

Добавим в semantics нужную логику: при вызове кастомного действия явно свайпнуть эту ячейку. И вот что получается.

```
Column(
  modifier = Modifier
        .semantics(mergeDescendants = true) {
            customActions = listOf(
                CustomAccessibilityAction(deleteString) {
                    scope.launch {
                        state.dismiss(DismissDirection.StartToEnd)
                    }
                    true
                }
            )
        }
        .fillMaxWidth()
        .padding(16.dp)
)
```



Habr
Самое интересное по вашим хабам

Habr, Самое интересное по вашим хабам, Window a11yforcompose

Особенности Accessibility в Compose

В отличие от системы View, поддержка Accessibility в Compose немного урезана:

- В роль теперь нельзя задать любую строку — только одну из заранее предопределённых: Button, Checkbox, RadioButton, Tab, Image и Switch.
- Нельзя отслеживать Accessibility-события у компонентов. Например, теперь не получится через `AccessibilityDelegate` и перегруженную функцию `performAccessibilityAction` перехватывать события. Сам по себе `AccessibilityDelegate` никуда не пропал: об этом — ниже.
- В Compose нет аналога `ViewCompat.announceForAccessibility`.

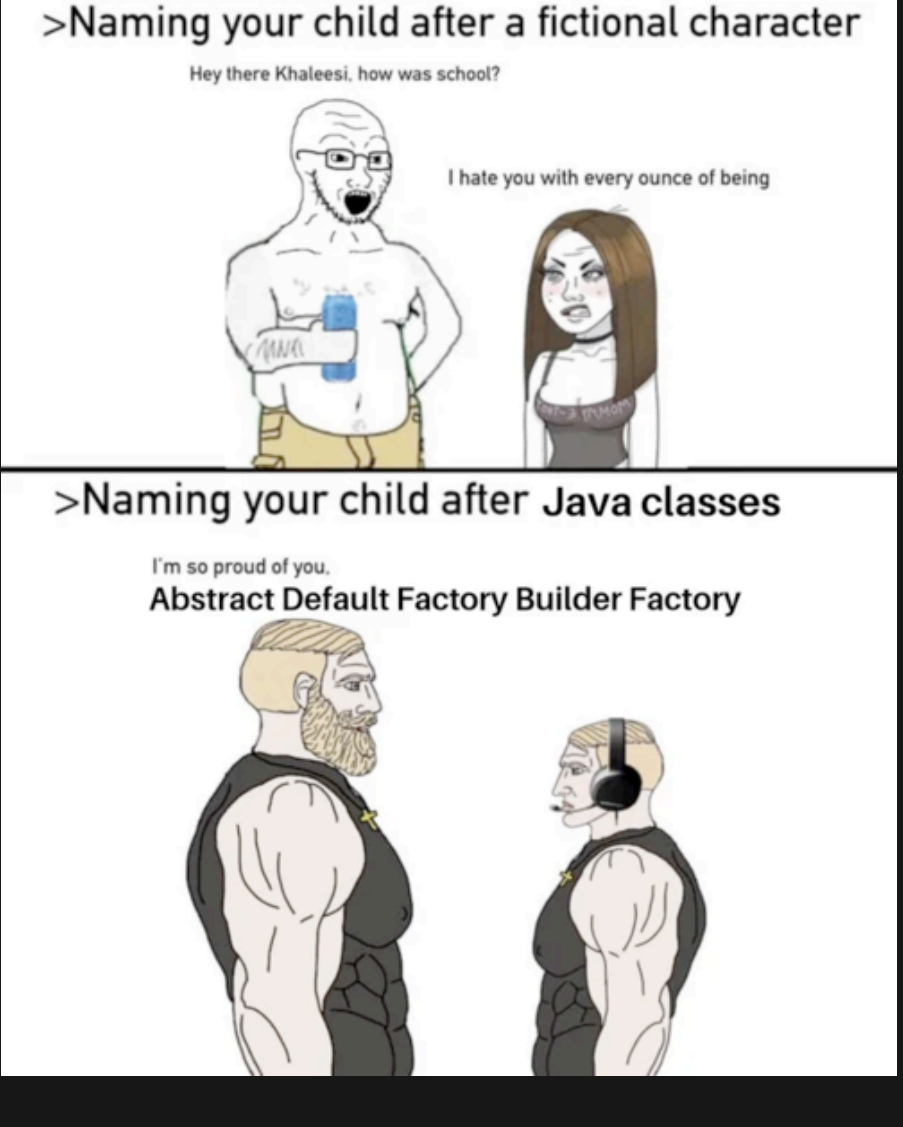
Это основные особенности, которые мне удалось обнаружить. Скорее всего, есть ещё что-то, чем обделили Compose. Если знаете, напишите в комментариях.

Эти изменения были сделаны не просто так: неправильное использование функциональности могло ввести пользователей в заблуждение, а разработчики могли понавставлять себе палки в колёса.

Основная рекомендация: доработки по Accessibility должны быть минимальны, не нужно строить никаких велосипедов. Оставшиеся инструменты должны покрывать весь скоуп возможных проблем.

Куда всё-таки пропал AccessibilityDelegate

А он никуда и не пропадал! Весь Compose рисуется на `ComposeView`. Compose поддерживает Accessibility, значит, у `ComposeView` есть свой собственный делегат. И имя ему `AndroidComposeViewAccessibilityDelegateCompat.android.kt`. Название страшное, логика внутри тоже. Смысл делегата — адаптировать все указанные семантики в Compose для ОС Android.



>Naming your child after a fictional character
Hey there Khaleesi, how was school?
I hate you with every ounce of being
>Naming your child after Java classes
I'm so proud of you,
Abstract Default Factory Builder Factory

Что в итоге

При разработке Jetpack Compose Гугл проделал огромную работу. Теперь намного легче сделать приложение доступным: не надо использовать страшные сущности типа `AccessibilityDelegateCompat` и перегружать в этом делегате странные методы. Подход стал проще: **пишешь код** как пишешь, и **оно работает**.

> Больше полезного про Android — в нашем телеграм-канале Surf Android Team. Здесь мы публикуем кейсы, лучшие практики, новости и вакансии Surf, а также проводим прямые эфиры. **Присоединяйтесь!**

Больше кейсов команды Surf **ищите на сайте >>**

Теги: surf, jetpack compose, android, мобильная разработка, accessibility, доступность, разработка под android, surfstudio

Хабы: Блог компании Surf, Программирование, Разработка мобильных приложений, Разработка под Android, Accessibility

Редакторский дайджест
Присылаем лучшие статьи раз в месяц
Электропочта

Surf

Создаём веб- и мобильные приложения

Сайт ВКонтакте Instagram Telegram Telegram Telegram Telegram Telegram

64 Карма **23** Рейтинг

@Surf_Studio

Пользователь

Подписаться

Комментарии

Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

JordanCpp 16 часов назад

Хабр таки торт

Простой 3 мин 4.5K

Мнение

+56 7 40 +40

popski_ruvds 22 часа назад

Жизнь и смерть Карла Коха: первый хакер на службе КГБ в поисках всемирного заговора и тайны числа 23. Часть 1

Простой 7 мин 4.3K

Ретроспектива

+32 30 4 +4

GlobalSign_admin 17 часов назад

Вредоносный код навсегда сохранили в блокчейне

4 мин 22K

+31 42 25 +25

the_bat 3 часа назад

Разработка диагностического кабеля VAG-KKL для работы с программой CASCADE

4 мин 3.5K

+30 8 2 +2

RationalAnswer 6 часов назад

Всё, что вам нужно знать о китайском DeepSeek в мемах, а также другие новости технофинансов

11 мин 11K

Дайджест

+24 7 15 +15

RyabovA 2 часа назад

Российское сборочное производство электроники с нуля. Или как делают одноплатники Repka Pi

Простой 14 мин 2K

Репортаж

+23 12 10 +10

Beeline_tech 19 часов назад

Семь самых важных научных статей в истории Computer Science

6 мин 4.7K

Дайджест Перевод

+23 79 9 +9

tigroid3 5 часов назад

Секреты стройности монолита: подходы по снятию нагрузки с БД

Простой 16 мин 1.5K

+20 22 5 +5

ivan_fr 4 часа назад

Observability vs Monitoring: почему в 2025 году это две стороны одной медали

Простой 8 мин 523

Обзор

+14 10 0

Dmitrii43 5 часов назад

Wiren Board как швейцарский нож: от выращивания страусов до цеха металлообработки

Простой 11 мин 899

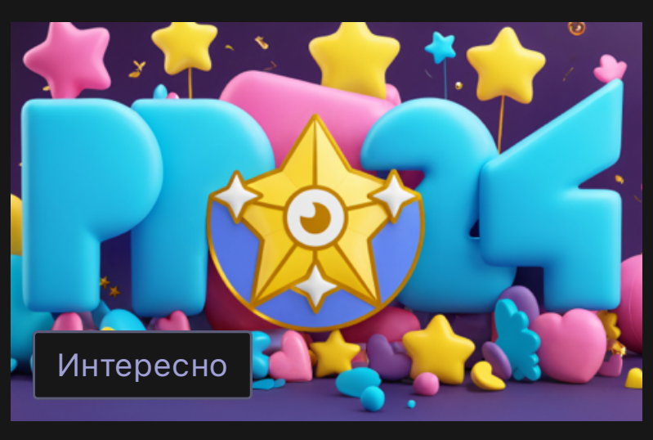
+14 9 5 +5

Роботяги: что будет делать ИИ в промышленности к 2035 году

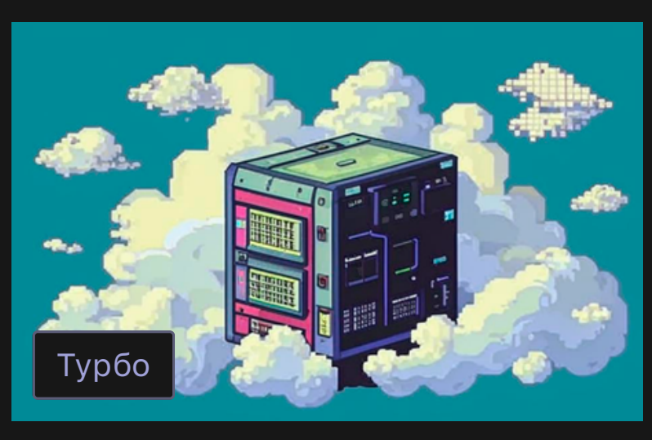
Турбо

Показать еще

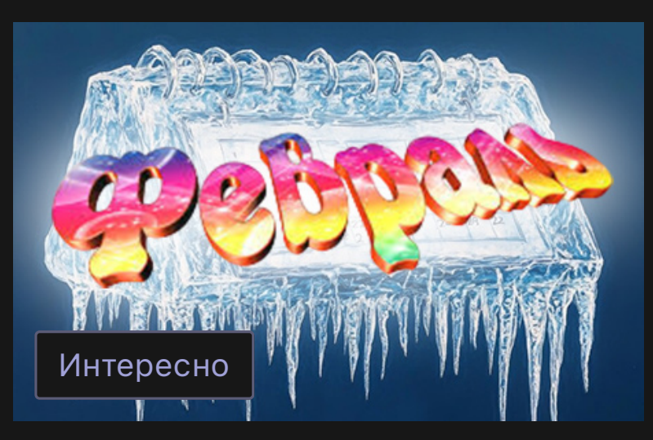
МИНУТОЧКУ ВНИМАНИЯ



Рейтинг лучших IT-работодателей 2024



Гиперконвергентная среда: OpenStack или VMware?


Февраль
Интересно

Зажигательные IT-ивенты, чтобы растопить вечную мерзлоту

КУРСЫ

Онлайн-курс Product Manager IT-проектов
7 февраля 2025 · Merion Academy

Фронтенд-разработчик с нуля
7 февраля 2025 · Merion Academy

Python - программист с нуля
7 февраля 2025 · Merion Academy

Онлайн курс по Linux
7 февраля 2025 · Merion Academy

Курс по этичному хакингу. Белый хакер
7 февраля 2025 · Merion Academy

Больше курсов на Хабр Карьере

РЕКЛАМА
Selectel
Облако Selectel бесплатно на 1 месяц
Получите до 30 000 бонусов на тест облачной платформы

ИНФОРМАЦИЯ

Сайт	surf.ru
Дата регистрации	4 июля 2019
Дата основания	29 января 2011
Численность	201–500 человек
Местоположение	Россия

ССЫЛКИ

Surf Flutter Team
t.me

Surf Tech
t.me

Surf Android Team
t.me

Surf iOS Team
t.me

Surf BA Team
t.me

Surf Web Team
t.me

Точка приложения
t.me

ВИДЖЕТ

Surf
Пишем про мобильную разработку и IT-карьеру в Telegram-канале surf_tech
Перейти в канал

Ещё больше контента в нашем Telegram-канале. Подписывайся.

БЛОГ НА ХАБРЕ

29 янв в 14:03
Не вручную: почему стоит автоматизировать рутинные задачи для QA
3.6K 1 +1

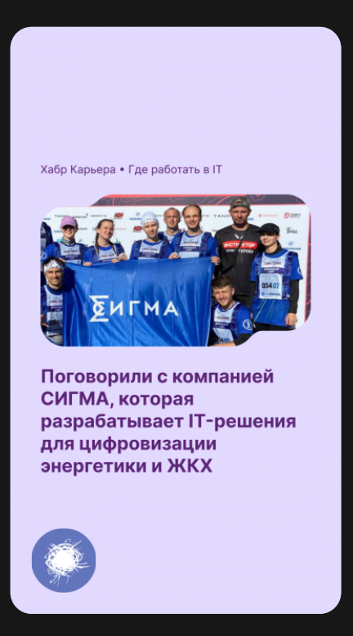
20 янв в 15:03
Архитектура для SwiftUI — VSURF
1.3K 8 +8

15 янв в 18:10
Суперaппы банков украдут ваших клиентов. Как сделать так, чтобы пользователи не ушли из ритейл-приложения
1.6K 4 +4

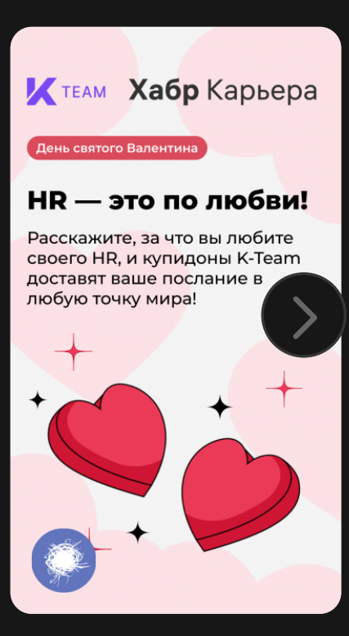
26 дек 2024 в 18:42
Surf-итоги кода: Flutter, Android и iOS в 2024-м
2.2K 1 +1

17 дек 2024 в 15:56
Всё, что нужно знать об Android 15: архивирование, защита данных и новые API
6K 6 +6

ИСТОРИИ


Хабр Карьера • Где работать в IT
СИГМА
Поговорили с компанией СИГМА, которая разрабатывает IT-решения для цифровизации энергетики и ЖКХ

Где работать в IT в 2025: СИГМА


K TEAM Хабр Карьера
День святого Валентина
HR — это по любви!
Расскажите, за что вы любите своего HR, и купидоны K-Team доставят ваше послание в любую точку мира!

Отправьте эйчару валентинку

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам

© 2006–2025, Habr

Техническая поддержка Настройка языка